Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Projeyle ilgili sorular.
- Gereksinim mühendisliğinin 7 adımının tekrarı (önceki dersin konusu). SRS belgesi ve izlenebilirlik tablosu.
- Nesneye yönelik çözümleme: alan (domain) analizi, kullanım öyküsü ve kullanım senaryosu belgesi.
- Use case diyagramı: gösterim, include ve extend ilişkileri.
- Sınıfların ve etkileşimlerin belirlenmesi. Kavramsal sınıf diyagramı (alan modeli).
- Analiz ve tasarım aşamasında hazırlanan belgeler.
- Sıralama (sequence) diyagramı ve tasarım kalıplarıyla birlikte kullanımı.
- Etkinlik (activity) diyagramı.
- Tasarım sınıf diyagramı.
- Sınıflar arası ilişkiler: bağlantı (association), kümeleme (aggregation), oluşum (composition), genelleştirme (generalization), çok şekillilik (polymorphism).
- Kahve satış sitesi örneği: use case diyagramı, senaryo belgesi, kavramsal ve tasarım sınıf diyagramları.

Hocanın Özellikle Vurguladığı Kısımlar

- **Projelerde izlenebilirlik tablosu (traceability matrix) olmalı:** Hangi gereksinimin hangi modülle, hangi sınıfla, hangi veri tabanı tablosuyla karşılandığı gösterilmeli.
- **SRS raporu** IEEE 830 ve 2011 tarihli ISO/IEC/IEEE standardı dikkate alınarak, en azından özetlenerek hazırlanmalı.
- **Her aktörün en az bir senaryosu olmalı.** Hiçbir senaryoyla ilişkilendirilemeyen aktör diyagramdan çıkarılır.
- **Kavramsal sınıf diyagramında işlemler (metotlar) yazılmaz.** Yalnızca sınıf adı, öznitelikler ve sınıflar arası ilişkiler gösterilir. İşlemler tasarım sınıf diyagramında eklenir.
- **Tasarımda en az iki diyagram şarttır:** sıralama diyagramları ve tasarım sınıf diyagramı. Bunlara etkinlik ve durum geçiş diyagramları da eklenmeli.
- **Projede işbirliği (collaboration) diyagramı kullanılmayacak.** Sıralama diyagramı tercih ediliyor, çünkü işlemlerin sırasını gösteriyor.
- **Diyagramlar mutlaka bir araçla çizilmeli** (Violet önerildi), standartlara uygun olmalı.
- **Sınavda bir paragraf verilip bu diyagramlardan herhangi biri istenebilir.** Hangi diyagramın analizde, hangisinin tasarımda çizildiğini ve aralarındaki bağlantıyı bilmek önemli.
- **Projede tüm sistemin analizi ve tasarımı yapılacak.** Yalnızca bir modül çalışır halde kodlanacak. Çalışan modülün kullanıcıdan girdi alan bir arayüzü olmalı.

Kısa Tekrar Notları

- **Gereksinim mühendisliğinin 7 adımı (Pressman):**
  1. Başlangıç (inception).
  2. Bilgi toplama (elicitation).
  3. İşleme/modelleme (elaboration).
  4. Pazarlık (negotiation).
  5. Tanımlama (specification).

6. Doğrulama (validation).
7. Gereksinim yönetimi (requirements management).

- **SRS (Software Requirements Specification):** Gereksinim analizinin çıktısı olan ayrıntılı gereksinim belgesi.
- **Bilgi toplama yolları:** yüz yüze görüşme (interview), anket, yerinde inceleme.
- **Paydaşlar:** müşteriler, yönetim, sponsor, uç kullanıcılar, yazılım geliştiriciler.
- **Kullanım senaryosu belgesi:** senaryo adı, birincil aktör, ilgililer ve ilgi alanları, ön koşullar, son koşullar, ana senaryo (sıralı adımlar), alternatif senaryolar.
- **Use case diyagramı gösterimi:**
  - Elips: senaryo/işlem.
  - Çöp adam: insan aktör.
  - Dikdörtgen: insan olmayan aktör (yazılım, donanım, sensör).
- **«extend»:** B, A'nın isteğe bağlı genişlemesidir. A yürütülürken koşula bağlı olarak oluşabilecek bir sapma.
- **«include»:** A gerçekleştiriliyorsa C de mutlaka onun içinde gerçekleşir.
- **Analiz belgeleri:** use case diyagramları, kullanım senaryo belgeleri, kavramsal sınıf diyagramı, gerekirse ER diyagramı.
- **Tasarım belgeleri:** sıralama diyagramları, tasarım sınıf diyagramı, etkinlik diyagramları, durum geçiş diyagramları.
- **Kümeleme (aggregation):** Parçalar kendi başına da var olabilir (içi boş baklava; gösterim büyük olasılıkla slayttan).
- **Oluşum (composition):** Parçalar yalnızca bütün için vardır, bütün yok edilince parçalar da yok olur (içi dolu baklava; gösterim büyük olasılıkla slayttan).
- **Genelleştirme:** Ana sınıfla alt sınıflar arasındaki ilişki. İçi boş üçgenle gösterilir.
- **Çok şekillilik (polymorphism):** Farklı sınıflardan yaratılmış nesnelerin aynı mesaja farklı tepki vermesi.

Detaylı Açıklamalar

Projeyle İlgili Sorular

- **Projeyle ilgili cevaplar:**
  - Üç konudan yalnızca biri seçilir.
  - Seçilen konunun tüm modüllerinin analizi ve tasarımı yapılır. Yalnızca bir modül sonuna kadar çalışır hale getirilir. Örnek: insan kaynakları sisteminde işe alım ya da bilgi güncelleme.
  - Uygulama nesneye yönelik çalışmalı ve kullanıcıdan girdi alan bir arayüzü olmalı.
  - Medya takip projesinde ortam simüle edilebilir ya da gerçekten internetten haber taranabilir. İkisi de kabul.

Gereksinim Mühendisliğinin Tekrarı

- **Gereksinim analizi:** Projenin amaçlarını, boyutlarını ve etkilerini belirleme çalışması. Önce bir fizibilite çalışması da yapılır.
- **SRS (Software Requirements Specification):** Ayrıntılı gereksinim bilgilerinin dökümü.
  - IEEE 830, SRS'in standart yapısını verir.
  - 2011'de ISO, IEC ve IEEE ortak çalışmasıyla daha kapsamlı bir standart oluşturuldu.
- Sistem analizi dersinde gereksinim analizi 3 adımdı: bilgi toplama, modelleme, belge oluşturma. Burada 7 adım var. Yazılım ekibi bu aşamada müşteriyle birlikte çalışır.

Adım	Yapılanlar
Başlangıç (Inception)	Bu işe neden giriyoruz? Gerekçelendirme (justification), fizibilite sonuçları.
Bilgi toplama (Elicitation)	Görüşme, anket, yerinde inceleme. Tüm paydaşların katılımı. Toplantıya hazırlık ve katılım kuralları.

Adım	Yapılanlar
İşleme (Elaboration)	Toplanan bilgilerin modellenmesi: kullanım senaryoları, senaryo belgeleri, kavramsal sınıf diyagramları.
Pazarlık (Negotiation)	Farklı paydaşların çelişen anlatımlarının açıklığa kavuşturulması. Geliştiricilerle kullanıcılar arasında fikir birliği.
Tanımlama (Specification)	Gereksinimlerin yazıya dökülmesi: doğal dilde belgeler, senaryolar, diyagramlar, matematiksel gösterimler, varsa prototip. SRS belgesi.
Doğrulama (Validation)	Gereksinimler açık mı, eksik ya da hatalı mı, gerçekçi mi? Pazarlıktaki değişiklikler yapılmış mı?
Yönetim (Management)	Gereksinim değişikliklerinin kuralları ve izlenmesi. İzlenebilirlik tablosu.

- **Pazarlık aşamasındaki sorular:** Her istek teknik olarak yapılabilir mi? Yapılabilse bile maliyeti çok mu yüksek? İstenenin ne kadarının yapılabileceğine bu dengeye göre karar verilir.
- **Gereksinim yönetimi:** Geliştirme sürecinde yeni gereksinim ekleme, değiştirme ya da öncelik sırasını değiştirme kurallarının önceden, resmi bir sözleşmeyle belirlenmesi istenir.
- **İzlenebilirlik tablosu (traceability matrix):** Gereksinimler modellemede, tasarımda ve kodlamada takip edilir. Her gereksinimin hangi modülle, hangi pakette, hangi sınıfla, hangi veri tabanı tablosuyla sağlandığı gösterilir. Projelerde istenecek.
- **Bilgi toplama ilkeleri:**
  - Gereksinimler hakkında ayrıntılı bilgi tüm paydaşların katılımıyla elde edilir.
  - Paydaşlar: sistemden doğrudan etkilenenler, müşteriler, yönetim, sponsor, uç kullanıcılar, yazılım geliştiriciler. Hepsinin bakış açısı farklıdır.
  - Yerinde inceleme: Daha fazla ayrıntı gerektiğinde, örneğin 3-4 gün boyunca sürecin içinde bulunarak izlenir.

Nesneye Yönelik Çözümleme

- **Amaç:** Sistemi anlamak, sistemdeki parçaları ve temel elemanları belirlemek.
- **Alan (domain) analizi:**
  - Uygulama alanı ve kavramları anlaşılır. Örneğin öğrenci bilgi sisteminde hangi işler yapılır, hangi terimler kullanılır?
  - Benzer uygulamalar incelenir. Örneğin başka üniversiteler hangi sistemleri kullanıyor?
  - Anket yapılır. Var olan sistemin hatalı ya da zor kullanılan tarafları belirlenir.
  - Uzman görüşü alınır.
  - Sonra gereksinimler ve sınıflar belirlenir.

Örnek (Larman'ın NextGen POS perakende satış sistemi): Müşteri isteklerini doğal dille anlatır:

- Eski yazılım ihtiyaçlarımızı karşılamıyor. Donanımı da yenileyerek satış programımızın yeniden yazılmasını istiyoruz.
- Program kasadaki alışverişe yardımcı olmalı. Her işlem saklanmalı.
- Mali bilgiler harici bütçe sistemine, mal çıkış bilgileri envanter sistemine iletilmeli.
- Saklanan işlemler üzerinde rapor ve analiz yapılabilmeli.
- Müşteriye fiş verilmeli. Her satışta KDV ayrıca hesaplanıp belirtilmeli.
- Birden fazla şube varsa merkezi sunucu kullanılabilir.

Kullanım öyküsü: Bu doğal dil paragrafını, programın yapacağı işi ayrıntılı ve kurallı anlatan bir biçime dönüştürmek.

- **Amaçları:**
  - Ürünün beklenen işlevlerini ve çalışma ortamını belirlemek.

  - Son kullanıcı ile yazılım ekibi arasında anlaşma zemini oluşturmak.
  - Sistemle kullanıcıların birbirini nasıl etkileyeceğini tanımlamak.
  - Kabul testlerini (acceptance tests) kolaylaştırmak.
- **Bölümleri:** giriş bölümü, ana senaryo, alternatif senaryolar.

Örnek senaryo belgesi (satış işlemi):

```
Senaryo adı      : Satış işlemi
Birincil aktör   : Kasiyer
İlgililer        : Kasiyer, satıcı, müşteri (ve ilgi alanları)
Ön koşul         : Kasiyer kimliğini doğrulamış olmalı
Son koşul        : Ödeme tahsil edilmiş olmalı
Ana senaryo      : 1. Müşteri alacağı ürünlerle kasaya gelir.
                   2. Kasiyer satış işlemini başlatır.
                   3. ... (algoritma adımları)
Alternatif akış  : 3a. Geçersiz barkod: sistem uyarı verir, kaydı reddeder.
                   3b. Müşteri bir kalemi çıkarmak isterse: kasiyer barkodu
                       okutur, sistem kalemi çıkarıp güncel toplamı gösterir.
```

- Kullanım öyküsü alan modelinin (domain model) kapsamı dışına çıkmamalı.
- Aktör insan olabileceği gibi yazılım ya da donanım gibi bir varlık da olabilir.

Use Case Diyagramı

- **Gösterim:**
  - Elips: her bir işlem/senaryo.
  - Çöp adam: insan aktör.
  - Dikdörtgen: insan olmayan aktör (yazılım, donanım, sensör).
  - Çizgiler: aktörlerle senaryolar arasındaki etkileşim.
- **Senaryolar arası ilişkiler:**
  - **«extend»:** Use case B, use case A'nın genişletilmişidir. B, A yürütülürken oluşabilecek bir sapma ya da alt akıştır. Mutlaka gerçekleşmez.
  - **«include»:** Use case C, A tarafından içerilir. A gerçekleştiriliyorsa C de gerçekleşir.
  - Senaryolar arasında özelleştirme (genelleştirme) de olabilir.
- Ok yönü aynı zamanda ilişkinin okunma yönüdür.
- **Bir aktör başka bir aktörle doğrudan etkileşimde gösterilmez.** Aralarında etkileşimi sağlayan bir senaryo olmalıdır. (Bu kural altyazıda geçmiyor, büyük olasılıkla slayttan.)

Sınıfların ve Etkileşimlerin Belirlenmesi

- **Sınıfları belirlerken:**
  - Hangi sınıflar var, hangi eylemler yapılıyor, sınıfların sorumlulukları ne?
  - Sorumluluklar genelden özele dağıtılır (kalıtım hiyerarşisi).
  - Bilgi ve o bilgiyle ilgili davranış aynı sınıfta olmalı (kapsülleme, encapsulation).
  - CRC (Class-Responsibility-Collaborator) kartları akılda tutulmalı.
- **Etkileşim:** Bir nesnenin sorumluluğunu yerine getirmek için başka bir nesneye mesaj göndermesi.
- Sınıf diyagramında nesneler arası bağlantılar, özelleştirme ve genelleştirme ilişkileri gösterilir.

Örnek alan modeli (kavramsal sınıf diyagramı, POS örneği; ayrıntılar büyük olasılıkla slayttan):

```
SatışKalemi (miktar)     --- içerir ---> Ürün (tanım, fiyat, kdv, barkod)   [* .. 1]
Satış (tarih)           <>--- 1..* SatışKalemi                             (oluşum)
Satış                   --- içerir ---> Ödeme (miktar)                     [1 .. 1]
Kasa                     --- kaydeder --> Satış
Şube (ad, adres)         --- stoklar ---> Ürün
Şube                    <>--- Kasa                                         (oluşum)
```

- Bir satışta bir ya da birden fazla satış kalemi olabilir. Her satış kaleminde bir ürün tanımı vardır.

- Kavramsal sınıf diyagramında sınıfın üçüncü bölmesi (işlemler) boştur.

Analiz ve Tasarım Belgeleri

- **Analizde mutlaka olması gerekenler:**
  - UML use case diyagramları.
  - Kullanım senaryo belgeleri.
  - UML (kavramsal) sınıf diyagramları.
  - Veri tabanı tanımı gerekiyorsa ER diyagramı.
- Bazı kaynaklar etkinlik diyagramlarını da analizde çizer. Bu derste daha çok tasarımda sayılıyor.
- **Tasarım:**
  - Sorumluluklar belirlenir. Kullanılacak tasarım kalıpları belirlenir.
  - Sıralama (ardışıl) diyagramları çizilir. Sıralama diyagramı analizde başlar, tasarımda ayrıntılanır.
  - Yazılım (tasarım) sınıf diyagramı çizilir.
  - Etkinlik ve durum geçiş diyagramları eklenir.
- **Tasarımın sonunda:** Sınıflar, aralarındaki etkileşimler, işbirlikleri ve bağlantılar belirlenmiş olur. Yalnızca metotların algoritmaları belli değildir. Diyagramlara programcıya ipucu veren notlar eklenebilir.
- **Kodlamada:**
  - Sınıf tanımları sınıf diyagramına bakılarak yapılır.
  - Metotlar sıralama diyagramlarına bakılarak yazılır.
  - Sonunda her sınıfta hangi işlemin nasıl gerçekleştiği kontrol edilir.

Sıralama (Sequence) Diyagramı

- Diğer adları: ardışıl diyagram, sıralama şeması.
- **Yapısı:**
  - Nesneler, hangi sınıftan oldukları belirtilerek, işleme girme sırasına göre en üste dizilir.
  - Aşağı doğru zaman akar. Nesnelerin işleme giriş sırası ve aralarındaki mesajlar gösterilir.
  - Mesaj bir metodun çağrılmasıdır. Geri dönüş kesikli çizgiyle gösterilir.
  - Nesne kendine mesaj gönderebilir.
  - Koşullu mesaj (if) ve döngü gösterilebilir.
  - Etkinlik çubuklarının uzunluğu önemli değildir. Bir sınıfın nereden itibaren işleme girdiğini gösterir.
- Mesajlar, tasarım sınıf diyagramındaki işlemlerdir.
- **İşbirliği (collaboration) diyagramı** sıralama diyagramının eşdeğeridir. Bu derste kullanılmayacak.

Örnek: Kahve sitesinde sepete kahve ekleme (slayttaki diyagram)

```
Nesneler: :Denetçi | :KahveKatalog | :Map<Kahve> | :Sipariş | sk:SiparişKalemi | siparişKalemleri

1: kahveEkle(id, miktar)        --> :Denetçi                      [Denetçi kalıbı]
2: tanim = tanimAl(id)          :Denetçi --> :KahveKatalog        [Uzman kalıbı]
3: tanim = bul(id)              :KahveKatalog --> :Map<Kahve>
4: siparişEkle(tanim, miktar)   :Denetçi --> :Sipariş
5: yarat(tanim, miktar)         :Sipariş --> sk:SiparişKalemi   [Yaratıcı kalıbı]
6: ekle(sk)                     :Sipariş --> siparişKalemleri
```

- Diyagram üzerinde hangi adımın hangi tasarım kalıbını (Denetçi/Controller, Uzman/Information Expert, Yaratıcı/Creator) kullandığı notlarla gösterilmiş.
- Özel kahve, birden fazla kahvenin karıştırılmasıyla oluşan bir siparişse, sipariş kalemlerinin eklenmesi bir döngü (loop) içinde gösterilir.
- Tasarım kalıpları sonraki derste ayrıntılı işlenecek.

Etkinlik (Activity) Diyagramı

- Akış diyagramına benzer. Özellikle işlemlerin aynı anda farklı ortamlarda (kulvarlarda) nasıl yürüdüğünü göstermek için kullanılır.
- **Elemanlar:**
  - Etkinlik başlangıcı ya da sinyal alma işareti. Akış sinyal gelene kadar bekler. Zamanla başlayan olaylar böyle gösterilir.
  - Eylem (etkinlik).
  - Karar (if yapısı).
  - Eşgüdüm çizgisi: çatallanma (fork) ve birleşim (join).
  - Etkinlik sonu.
- **Eşgüdüm:**
  - Birleşimde: Çizgiye varan akış, diğer akışların hepsi gelene kadar bekler, sonra tek akış olarak devam eder.
  - Çatallanmada: Tek akış, çizgiden sonra farklı etkinliklere ayrılır.
- **Örnek:** Uçağa biniş işlemleri. Yer hizmetleri ve müşteri hizmetleri farklı kulvarlarda, kısmen aynı anda yürür.
- Durum (state) diyagramları sonraki derste işlenecek.

Tasarım Sınıf Diyagramı (Kahve Sitesi)

- Kavramsal sınıf diyagramından farkı: Her sınıfın gerçekleştirdiği işlemler (metotlar) de yazılır.
- Kaynak: Hocanın Göksel Biricik ile yazdığı kitaptaki kahve sitesi örneği (Şekil 12.14).

Sınıf	Öznitelikler	İşlemler
KahveSitesi	adres: String, isim: String	
KahveKatalog		tanimAl()
Kahve	tanim: String, fiyat: Money, id: int	bul()
Denetçi		kahveEkle()
Sipariş	tamamlandı: boolean	tamamlan(), siparişEkle(), ödemeYap(), toplamAl(), faturaBas()
SiparişKalemi (soyut)	miktar: int	
StandartKalem	stdFiyat: Money	altToplamAl()
ÖzelKalem	newAttr: int	altToplamAl()
Fatura	id: int, tarih: Date	
Ödeme	miktar: Money, tarih: Date	

- **İlişkiler:**
  - KahveSitesi → KahveKatalog (1), KahveKatalog → Kahve (1..*).
  - KahveSitesi → Denetçi (1), Denetçi → Sipariş (1..*).
  - Sipariş → SiparişKalemi (1..*), SiparişKalemi → Kahve (1).
  - Sipariş → Fatura (1), Sipariş → Ödeme (1).
- **Genelleştirme:** StandartKalem ve ÖzelKalem, SiparişKalemi'nden türer. İkisinde de miktar vardır.
  - Standart kalemde tek ürünün fiyatı kullanılır.
  - Özel kalemde birden fazla ürünün fiyatı birleştirilerek alt toplam bulunur.

Sınıflar Arası İlişkiler

- **Bağlantı (association):** İki sınıf arasındaki ilişki. Çokluk (multiplicity) belirtilir:
  - `0..1`: en az sıfır, en çok bir.
  - `1`: tam olarak bir.
  - `0..*`: hiç olmayabilir, varsa çok sayıda olabilir.
  - `1..*`: en az bir, çok sayıda olabilir. Örnek: Bir siparişte en az bir kalem olmalı.

- **Kümeleme (aggregation):** Parça-bütün ilişkisi. Alt parçalar yalnızca o sınıfa ait değildir, kendi başlarına da kullanılabilir. Örnek: Liste ile yığın arasındaki ilişki. Liste, yığına bağlı olmadan yaratılıp başka nesnelere de hizmet verebilir.
- **Oluşum/birleşim (composition):** Parçalar yalnızca o nesneyi oluşturmak için vardır, kendi başlarına kullanılamaz. Nesne yok edilince tüm parçaları da yok olur. Örnek: Kitap nesnesi yok edilince ünite ve bölümleri de yok olur.
- **Genelleştirme (generalization):** Ana sınıfla alt sınıflar arasındaki ilişki. İçi boş üçgenle gösterilir.
  - Örnek: Müşteri (ad, adres) genel sınıftır.
  - Kurumsal müşteride ek olarak kredi derecesi ve kredi limiti vardır.
  - Şahsi müşteride kredi kartı tipi vardır.
  - İşlemler müşteri tipine göre farklıdır.
- **Çok şekillilik (polymorphism):** Farklı sınıflardan yaratılmış nesnelerin aynı mesaja farklı tepki vermesi. Alt sınıf miras aldığı işlemi genellikle olduğu gibi kullanır. Farklı davranması gerekiyorsa işlemi yeniden tanımlar. Soyut sınıf kavramı bununla ilişkilidir.

Kahve Sitesi Use Case Örneği

- **Senaryo (bilgi toplama sonucu):**
  - Site kahve satışı yapar.
  - Müşteri hazır kahve alabilir ya da özel kahve siparişi verebilir. Bir alışverişte birden fazla kahve siparişi verilebilir.
  - Müşteri teslimat ve ödeme formlarını doldurur. Ödeme kredi kartı ya da banka havalesiyle yapılır. Müşteri bilgileri doğrulanır.
- **Adımlar:**
  1. Genel akış çıkarılır.
  2. Aktörler belirlenir: müşteri ve kahveyi sağlayan depo.
  3. Senaryolar belirlenir. Senaryo, bir aktöre fonksiyonel bir değer sunan bütündür.
- Aktörlerin mi senaryoların mı önce belirleneceği analistin tercihine kalmıştır.

Use case diyagramı (slayttaki Şekil 11.2):

```
Aktör: Müşteri
  - Kahve Seçimi
  - Özel Kahve Oluşturma  --«extend»-->  Kahve Seçimi
  - Bilgi Girişi
  - Sipariş Verme         --«include»--> Bilgi Girişi
  - Sipariş Teslimatı
Aktör: Depo
  - Sipariş Teslimatı     --«include»--> Fatura Basımı
```

- Müşteri hazır kahvelerden seçip özel kahve oluşturmaya hiç uğramadan da devam edebilir. Bu yüzden ilişki «extend».
- Sipariş verirken bilgi girişi yapılmış olmalı. Bu yüzden ilişki «include». (Bilgi girişi yalnız ilk siparişte yapılıyorsa bu durum büyük olasılıkla ön koşulla ya da «extend» ile daha doğru modellenir, bkz. ders sonrası notlar.)
- Senaryo belgesinde birincil aktör müşteri, ilgili de kahve sitesidir. Hoca, sitenin bir sistem olduğu için dikdörtgenle de çizilebileceğini söyledi.

Sonraki Hafta

- Durum (state) diyagramları ve tasarım kalıpları.

Ders sonrası notlar

- **SRS standardı:** IEEE 830-1998'in yerini büyük olasılıkla 2011'de ISO/IEC/IEEE 29148 aldı. Hocanın "2011'deki ISO, IEC ve IEEE ortak çalışması" dediği bu standarttır. Altyazıda "IEK" diye geçen IEC'dir.
- **Kayıt boşluğu:** Hocanın "geçen hafta" diye andığı gereksinim mühendisliği dersi (büyük olasılıkla 22 Mart 2023) bu şubenin kayıtlarında yok. Bu derste o konunun yalnızca tekrarı var.

- **Bilgi girişi örneğindeki ilişki:** Hoca bilgi girişi için “daha önce yapılmışsa buraya uğraması gerekmiyor” dedi. Bu, koşula bağlı bir davranıştır ve «include» tanımıyla tam örtüşmez. «include» edilen senaryo her seferinde çalışır. Eğer bilgi girişi yalnızca ilk siparişte yapılıyorsa «extend» ilişkisi ya da “bilgi girişi yapılmış olmalı” diye bir ön koşul daha doğru olur.
- **İşbirliği diyagramında sıra:** Hoca işbirliği diyagramının sırayı göstermediğini ima etti. UML 2’deki adıyla iletişim (communication) diyagramı da mesajları numaralandırarak sırayı gösterir. Sıralama diyagramı ise sırayı zaman ekseninde daha okunur gösterir. Hocanın kastı büyük olasılıkla nesnelerin zaman sırasına göre dizilmemesidir; derste sıralama diyagramının tercih edilmesinin nedeni budur.
- **Aktör gösterimi:** UML’de insan olmayan aktörler «actor» stereotipli bir dikdörtgenle gösterilebilir. Çöp adam da her tür aktör için kullanılabilir. Dikdörtgen bir zorunluluk değil, okunurluğu artıran bir tercihtir.
- **Tasarım kalıbı adları:** Slayttaki Denetçi, Uzman ve Yaratıcı, Larman’ın GRASP kalıplarıdır: Controller, Information Expert, Creator.
- **Altyazıdaki bozuk yerler:**
  - “bilgi toplama, illustration” = elicitation.
  - “Next Gen Post” = NextGen POS.
  - “birinci laktör” = birincil aktör.
  - “seconds/sekons diagram” = sequence diagram.
  - “artificial diagram” ve “artışıl diagram” = ardışıl diyagram.
  - “Kolobreysun” = collaboration.
  - “Yesoft / Y-Soft” proje konusundaki yazılım evinin adı.
  - “Beş Duyu Medya Takibi” medya takip konusunun tam adı olabilir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.